

MCP-Unity: Protocol-Driven Framework for Interactive 3D Authoring

Shutong Wu
Computer Science Department
University of Rochester
Rochester, New York, USA
swu85@ur.rochester.edu

Justin P. Barnett
CoPlay
San Francisco, California, USA
justin.barnett@hey.com

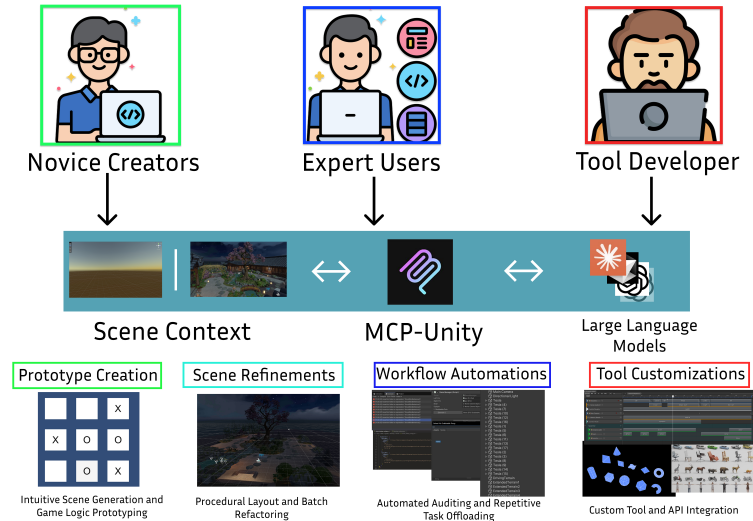


Figure 1: MCP-Unity leverages the Model Context Protocol to move beyond brittle code generation, transforming AI assistants into state-aware, reliable partners within Unity. By connecting a powerful AI to the project through a structured command set, our tool empowers a spectrum of users: novices can intuitively generate and modify scenes, experts can automate tedious workflows, and developers can build custom, AI-powered tools on an extensible foundation.

Abstract

Professional 3D authoring tools present a steep learning curve, and existing AI integrations that rely on code generation are often brittle and lack state awareness. We present MCP-Unity, a novel system enabling seamless integration between AI assistants and Unity through the Model Context Protocol. Instead of generating runtime code, the system constrains the Language Models to a structured command set, enabling a state-aware, interactive dialogue between user and AI assistants. This protocol-driven approach is implemented in a three-tier architecture designed to support a spectrum of users, from novices who benefit from an intuitive natural language interface to experts who can automate complex, repetitive tasks. Findings from a preliminary study with 17 novice teenage learners revealed high engagement and the emergence of concise, conversational interaction patterns, validating the system's

potential to lower the barrier to entry. MCP-Unity replaces fragile code generation with protocol-driven automation, enabling stable, extensible human-AI collaboration in Unity.

CCS Concepts

• **Human-centered computing** → **Interactive systems and tools**; **Virtual reality**.

Keywords

Game Technology, 3D Generation, Human-Computer Interaction, Software Engineering

ACM Reference Format:

Shutong Wu and Justin P. Barnett. 2025. MCP-Unity: Protocol-Driven Framework for Interactive 3D Authoring. In *SIGGRAPH Asia 2025 Technical Communications (SA Technical Communications '25)*, December 15–18, 2025, Hong Kong, Hong Kong. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3757376.3771417>

1 Introduction

The creation of 3D digital content remains a specialized skill, gated by the complexity of professional authoring tools like Unity [Unity



This work is licensed under a Creative Commons Attribution 4.0 International License. *SA Technical Communications '25, Hong Kong, Hong Kong*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2136-6/25/12
<https://doi.org/10.1145/3757376.3771417>

Technologies 2025]. These environments' complexity and learning curve present a barrier that hinders both novices and experts in their own domain. Large Language Models (LLMs) have emerged as a promising method to bridge this gap, but previous integration strategies rely on generating raw source code and lack the versatile interaction modalities, such as direct UI manipulation.

This coding-based approach, seen in systems like LLMR [De La Torre et al. 2024], can be error-prone and opaque, making it difficult for users to understand or modify the AI's actions easily. More fundamentally, a general-purpose LLM lacks deep, intrinsic knowledge of the target engine's specific information and constraints. This approach is ill-suited to the diverse needs of the creative community: it is too rigid for novices, too inefficient for experts, and too opaque for developers who wish to extend its capabilities.

We present MCP-Unity, a system that reframes the role of LLMs from a code generator to a protocol-driven agent under the Unity context. Instead of generating general-purpose code, MCP-Unity constrains the LLM to operate via a structured protocol of commands that map directly to Unity with engine-specific knowledge leveraging the Model Context Protocol (MCP) [Anthropic 2024]. By continuously providing the LLM with the current scene state, it makes grounded, transparent decisions by directly calling tools defined by protocol. This transforms the 3D authoring experience into an interactive, conversational, and reliable collaboration. The contributions of this work are twofold:

- (1) One of the first adaptations of the MCP framework on Unity for intuitive 3D authoring.
- (2) Opportunities and challenges gathered from the preliminary case study and the community feedback.

2 Related Work

2.1 AI for 3D Scene Generation

The effort to automate 3D interactive game creation with Artificial Intelligence (AI) has diverged into two primary branches: systems that generate entire, self-contained worlds, and systems that assist in manipulating content within existing frameworks.

- **Generative Models:** This branch focuses on synthesizing complete, interactive 3D environments starting from high-level prompts. Recent breakthroughs, such as WonderWorld [Yu et al. 2025] and Marble [Labs 2025], exemplify this approach. These trained models can use a single image or text description to generate a playable, action-controllable 3D world. While these systems show promise in creating novel experiences from scratch, they operate as self-contained ecosystems, often lack the fine-grained control necessary for detailed scene editing, and are computationally expensive when generating high-resolution, complex scenes [Yu et al. 2025].
- **LLM-Based Agents:** This second branch aims to integrate LLMs as assistants within professional 3D authoring tools such as Unity [Unity Technologies 2025]. A prominent method involves having LLMs generate executable code that is compiled in real-time, often supplemented by asset retrieval from online repositories or generative APIs. LLMR [De La Torre et al. 2024] and GROMIT [Jennings et al. 2024] are two pioneering systems that generate codes at runtime to create and

modify interactive experiences in Unity, and DreamGarden [Earle et al. 2025] adapts a similar prompt-to-scene workflow to Unreal engine. However, their reliance on code generation and lack of fine-grained control perpetuates the gap between the user's intention and final result, causing a vast gulf of execution [Norman et al. 1986].

2.2 Model-Context-Protocol Framework

The Model-Context-Protocol (MCP) is a framework from Anthropic designed to standardize how AI models interface with external tools and data sources [Anthropic 2024]. It addresses a common integration challenge where projects independently develop bespoke solutions by providing a universal, client-server architecture. An MCP Host (the user-facing AI application) manages Clients that establish secure, one-to-one connections with MCP Servers. These servers expose their capabilities through a standard set of primitives: Resources (read-only data), Tools (executable functions), and Prompts (task templates). Specialized MCP servers are developed for various tasks, including community projects such as Blender-MCP [Ahuja 2025] and official servers such as Figma-MCP [Figma 2025]. Our work represents one of the first applications of this standard to create a real-time bridge for controlling a professional 3D game engine, selecting Unity for its widespread popularity among creators and comprehensive documentation.

3 MCP-Unity

MCP-Unity is our implementation of the MCP paradigm under Unity. The system prioritizes seamless, state-aware, and extensible connections with the Unity Editor.

3.1 Three-Tier Design

MCP-Unity employs a three-tier system consisting of:

- (1) **AI Client Layer:** LLMs with MCP protocol support.
- (2) **Python MCP Server:** Protocol adaptation, connection management, and logic orchestration.
- (3) **Unity C# Bridge:** In-engine command execution, scene context retrieval, and native API access.

This decoupled approach ensures the Unity Editor remains responsive in general situations. The Python-based MCP server provides tool templates and descriptions to guide the LLM's decision-making process. The Unity MCP Bridge focuses on two core tasks: gathering scene and project context for the LLM, and executing validated protocol commands. When a user types a command such as "Generate a red ball", the context is bundled and sent to the server. A response in the protocol format is received and executed within seconds, first generating a sphere primitive, naming it "RedBall", and then generating or finding a red material and applying it to the ball. The supported tools can be found at Table. 1.

3.2 Protocol Design Principles

The core engineering challenge lies in mapping Unity's extensive API to a command-based protocol that LLMs can reliably utilize. We established four design principles through iterative development:

- (1) **Categorical Organization:** Unity's API surface encompasses thousands of functions, each on a different layer with

Table 1: Unity-MCP Tool Categories and Capabilities

Tool Category	Primary Capabilities
Scene Management	Hierarchy manipulation, lighting setup
GameObject Control	Transform operations, component management
Asset Pipeline	Import, creation, modification workflows
Script Management	C# file CRUD, compilation triggering
Shader Development	Shader creation, property binding
Editor State	Playmode control, selection management
Console Interface	Log retrieval, error monitoring
Menu Integration	Custom action execution

different uses. We organize these into eight logical categories (Scene, GameObject, Asset, Script, Shader, Editor, Console, Menu) that match common development workflows. This structure aids both user understanding and the LLM’s tool-calling process.

- (2) **Type-Safe Parameters:** Unity’s strongly-typed API requires careful parameter validation. Our protocol enforces type safety through JSON schema validation before execution, preventing runtime errors that corrupt the scene state.
- (3) **Context Preservation:** AI assistants require current scene state to make informed decisions, and the protocol includes context bundling (selected objects, scene hierarchy, recent changes, and log history) with each request and action made.
- (4) **Response Format:** Since failures are inevitable in complex interactions and code generation, the system provides structured error responses with detailed context instead of generic exceptions. This enables AI assistants to diagnose the cause of failure and potentially self-correct or suggest alternatives, a crucial feature for all users, especially novices.

4 User scenarios and preliminary case study

MCP-Unity is designed to provide distinct value across a spectrum of user expertise. We frame its contribution by addressing the needs of three key personas [Pruitt and Grudin 2003]: the Novice Learner, the tool developer, and the Expert Practitioner. Table.2 summarizes these roles we identified and their primary usages.

4.1 Preliminary User Study

We conducted a preliminary user study with high school and middle school learners to evaluate our system in practice. Participants (N=17; ages 14–17; none had prior Unity experience) worked in 8 pairs. Each pair was instructed to create a themed 3D scene (“city,” “car,” “garden,” etc.) using only text-based MCP commands. Participants were given a template sheet, a 10-minute warmup session, and a 30-minute creative task to build a 3D scene representing an abstract concept of their choice. We logged all commands and system responses and conducted post-session, semi-structured interviews.

4.2 Key Findings

High-Level Intent vs. System Capabilities. Participants often tried to recreate complex real-world scenes such as stadiums or vehicles. These requests exposed a fundamental mismatch: while

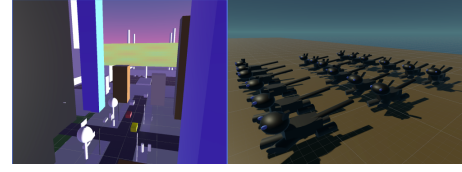


Figure 2: Students’ generation results through the user study, each went through iterative prompts. Left: Envision of Futuristic City; Right: Army of Robot Rabbits

MCP-Unity excels at tool orchestration and primitive geometric manipulation, it cannot perform complex 3D modeling. This often resulted in abstract or humorous approximations, such as a “basketball court” rendered as a flat plane with two cylinders.

Emergent Conversational Interaction Patterns. Participants exhibited remarkably concise communication, averaging 3.2 words per command (range: 1-8). Common patterns included imperative phrases like “make tree” or comparative adjustments like “bigger house”. This mirrors natural human communication, but provides less context to LLMs, resulting in a less aligned execution.

High Engagement Despite System Limitations. Despite frequent failures when requesting complex models, participants remained highly engaged. All 17 participants completed their creative tasks, with 12 continuing to iterate on their scenes after the session concluded. Post-session interviews revealed that the immediate visual feedback was highly compelling: “I like the modding activity and we built the stadium!” (P1 & P3).

Scaffolding Towards Deeper Tool Use. Participants expected near-instantaneous responses for commands (< 2s), and reported the long latencies (10-30s) for standard operations using MCP-Unity. This is a clear bottleneck limited by model response and single-threaded operation, which suggests a need for batch or sequence operations in future versions. Half of the participants expressed interest in learning more about Unity, with several asking if their scenes are saved and if they can continue modifying them afterward. This suggests that AI-assisted tools can serve as an effective scaffolding mechanism, lowering the initial barrier to entry and encouraging novices to learn authoring tools.

4.3 Implications

Our findings with novice learners validate the core premise of MCP-Unity for this user group: it successfully democratizes basic scene composition and fosters high engagement. The discrepancy between user expectations for 3D modeling and the system’s capabilities highlights the critical need for the integration of asset generation and retrieval models. The emergent conversational style strongly indicates that future interfaces should prioritize context-aware, low-verbosity commands over formal, rigid syntax, especially for novices.

5 Discussions

5.1 Findings through Community Development

Our project has been open-sourced since the start, and we have been receiving numerous feedback and support over the past six months. The iterative development offers several key insights:

Table 2: Target User Personas for MCP-Unity

Persona	Core Need	MCP-Unity Solution	Example Use Case
Novice User	Lowering the barrier to entry; translating creative ideas into 3D scenes without prerequisite technical knowledge.	An intuitive, natural language interface for scene creation and modification.	“Create a forest scene with ten pine trees and a small lake. Make the lighting look like a sunset.”
Expert User	Accelerating workflows by automating tedious, repetitive, or labor-intensive tasks.	Complex scene queries, and rapid, non-destructive prototyping. Offloads cognitive load from laborious processes.	“Find all GameObjects with the tag LOD1, Find the common properties associated with these GameObjects.”
Tool Developer	Extending the editor; creating new AI-powered tools and integrating custom production pipelines.	The protocol-driven design provides a stable and predictable API target compared to raw code generation.	Developing a new tool that uses MCP-Unity to generate stylization shaders procedurally.

- (1) **Protocol Modularity Matters:** The eight-category organization proved robust as new Unity features emerged. Adding support for new features and functions only requires extending existing categories rather than redesigning the protocol.
- (2) **AI-Game Collaboration beyond Prototyping:** While our initial study focused on novices, the architectural principles of MCP-Unity are designed for users, including experts and tool developers. AI-Game collaboration should be more than a simple prompt of “Generate a flower there”, and extend to tasks such as debugging, optimization, and batch processing.
- (3) **Beyond the Tool:** One of the most common usage for this tool is asset generation, which is beyond the scope of the tool, as asset generation would require external tools such as Meshy [Meshy, Inc. 2025] or Tencent-Hunyuan [Tencent Hunyuan3D Team 2025], which are either beyond the open source territory or the computing power limit. We are trying to collaborate with asset generation companies to incorporate these into our tool to create a more fulfilling user experience.

5.2 Limitations and Future Work

Current limitations include single-instance Unity support due to port binding constraints and limited coverage of Unity’s specialized APIs (Timeline, Cinemachine, Input System). We also lack a direct comparison with traditional Unity workflows, thus a controlled evaluation will be part of future work to better quantify how the system lowers the learning barrier. The personas were gathered through community feedback with no official studies, so we are currently planning follow-up studies with Unity developers and tool authors to evaluate usability and productivity gains.

Future work will tackle these limitations and explore the multi-user collaborative scenarios where multiple AI assistants coordinate scene modifications. We are also exploring incorporating additional inputs such as voice and gestures, which are useful for applications leveraging Augmented and Virtual Reality.

6 Conclusion

We present MCP-Unity, a system that explores a stable, interactive, and extensible human-AI collaboration in Unity through the Model Context Protocol architecture. Our structured approach provides a

middle ground between error-prone code generation and expert-only scripting interfaces, thus making the tool suitable for various creators, such as novices, tool developers, and experts. Our preliminary study confirms high engagement and intuitive interaction among new users. Our open-source implementation¹ provides a foundation for future research in AI-Game integration and interactive 3D authoring.

Acknowledgments

We thank David Sarno and the CoPlay team for their ongoing support and collaboration.

References

- Siddharth Ahuja. 2025. Blender-MCP: Bridging Blender and Claude AI via Model Context Protocol. <https://blender-mcp.com/>
- Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>
- Fernanda De La Torre, Cathy Mengying Fang, Han Huang, Andrzej Banburski-Fahey, Judith Amores Fernandez, and Jaron Lanier. 2024. LLMR: Real-time prompting of interactive worlds using large language models. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–22.
- Sam Earle, Samyak Parajuli, and Andrzej Banburski-Fahey. 2025. DreamGarden: A Designer Assistant for Growing Games from a Single Prompt. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–19.
- Figma. 2025. Figma Dev Mode MCP Server: Structured Design Context via MCP. <https://www.figma.com/Dev-Mode-MCP-Server-lets-AI-agents-access-design-structures-in-Figma-via-Model-Context-Protocol-beta>
- Nicholas Jennings, Han Wang, Isabel Li, James Smith, and Bjoern Hartmann. 2024. What’s the Game, then? Opportunities and Challenges for Runtime Behavior Generation. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (UIST ’24). Association for Computing Machinery, New York, NY, USA, Article 106, 13 pages. doi:10.1145/3654777.3676358
- World Labs. 2025. MARBLE™: AI-Powered 3D Scene Generation and Editing Platform. <https://marble.worldlabs.ai/>
- Meshy, Inc. 2025. Meshy.ai: Instant AI-Powered 3D Model Generation.
- Don A. Norman, James D. Hollan, and Edwin L. Hutchins. 1986. Gulf of Execution. In *User-Centered System Design: New Perspectives on Human-Computer Interaction*, Don A. Norman and Stephen W. Draper (Eds.). Lawrence Erlbaum Associates.
- John Pruitt and Jonathan Grudin. 2003. Personas: Practice and Theory. In *Proceedings of the 2003 Conference on Designing for User Experiences (DUX ’03)*. ACM, 1–15. doi:10.1145/997078.997089
- Tencent Hunyuan3D Team. 2025. Hunyuan3D 2.1: From Images to High-Fidelity 3D Assets with Production-Ready PBR Material. arXiv:2506.15442 [cs.CV]
- Unity Technologies. 2025. Unity Real-Time Development Platform. <https://unity.com>
- Hong-Xing Yu, Haoyi Duan, Charles Herrmann, William T Freeman, and Jiajun Wu. 2025. Wonderworld: Interactive 3d scene generation from a single image. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 5916–5926.

¹<https://github.com/CoplayDev/unity-mcp>